

Tools do Claude Code

Guia de referência rápida

O que é uma tool: cada ação que o Claude executa no seu ambiente (ler arquivo, rodar comando, editar código) passa por uma *tool* com nome e parâmetros definidos. Isso importa em quatro lugares:

1. **Permissões** — as regras de allow/ask/deny referenciam tools (`Bash(git status:*)` , `Edit(src/**)`).
2. **Hooks** — o `matcher` de `PreToolUse` / `PostToolUse` casa com o **nome da tool** (é assim que se intercepta "todo Bash" ou "todo Edit").
3. **Subagentes** — o campo `tools:` do frontmatter é a allowlist do agente (enforcement de papel).
4. **Skills/commands** — `allowed-tools:` restringe o que pode rodar dentro deles.

1. As essenciais (o dia a dia)

Tool	O que faz	Muta?	Pede aprovação?*
<code>Read</code>	Lê arquivo (texto, imagem, PDF, notebook)	Não	Não (no diretório de trabalho)
<code>Glob</code>	Encontra arquivos por padrão (<code>**/*.ts</code>)	Não	Não
<code>Grep</code>	Busca conteúdo em arquivos (ripgrep)	Não	Não
<code>Write</code>	Cria arquivo ou sobrescreve por inteiro	Sim	Sim
<code>Edit</code>	Substituição exata de trecho em arquivo existente	Sim	Sim
<code>NotebookEdit</code>	Edita células de Jupyter notebook	Sim	Sim
<code>Bash</code>	Comando de shell	Depende	Sim, exceto comandos read-only reconhecidos (<code>ls</code> , <code>cat</code> , <code>git status/log/diff</code> , <code>grep</code> , <code>find</code> ...)
<code>PowerShell</code>	Shell nativo no Windows (opt-in em Linux/macOS)	Depende	Como o Bash
<code>WebFetch</code>	Baixa uma URL e extrai conteúdo (vira Markdown)	Não	Sim, exceto domínios de documentação pré-aprovados
<code>WebSearch</code>	Busca na web	Não	Sim
<code>Agent</code>	Cria um subagente com contexto próprio (era <code>Task</code> até a v2.1.63)	—	Não
<code>AskUserQuestion</code>	Pergunta de múltipla escolha ao usuário	—	Não
<code>TaskCreate</code> / <code>TaskUpdate</code> / <code>TaskList</code>	Lista de tarefas da sessão (substitui o <code>TodoWrite</code> quando ativa)	—	Não
<code>Skill</code>	Invoca uma skill (<code>/comando</code>)	—	Sim

* Comportamento no modo padrão (Manual). Os modos de permissão mudam isso — ver §5.

2. Qual usar quando (e por quê)

O Claude é instruído a **preferir as tools dedicadas ao Bash** — elas são mais seguras (read-only sem prompt), mais eficientes e têm melhor tratamento de contexto (paginação, truncamento explícito):

Em vez de...	Use	O que você ganha
<code>cat</code> no Bash	<code>Read</code>	Vê imagens/PDFs/notebooks; paginação <code>offset / limit</code> ; aviso explícito de truncamento
<code>grep / rg</code> no Bash	<code>Grep</code>	Modos de saída (<code>files_with_matches / content / count</code>), multiline, respeita <code>.gitignore</code> , paginação
<code>find</code> no Bash	<code>Glob</code>	Sintaxe simples, ordenado por mtime; (<code>find</code> ainda vale para queries complexas: idade, permissões, <code>-exec</code>)
<code>curl</code> no Bash	<code>WebFetch</code>	HTML → Markdown automático, cache de 15 min; (<code>curl</code> ainda vale para dados brutos/APIs)
Reescrever arquivo inteiro	<code>Edit</code>	Mudança cirúrgica: <code>old_string</code> precisa casar exatamente e ser único (ou <code>replace_all</code>) — falha barulhenta em vez de sobrescrita silenciosa
<code>Edit</code> para arquivo novo	<code>Write</code>	Criação/reescrita total; <code>Edit</code> exige leitura prévia do arquivo, <code>Write</code> cria do zero
Fazer tudo inline	<code>Agent</code>	Contexto separado: o subagente explora/executa e devolve só o resultado final — a conversa principal não incha

3. Tools e subagentes

- `tools: no frontmatter` = allowlist. Só o que está listado existe para o agente — é *enforcement* de papel, não sugestão (o analista sem `Edit / Bash` não escreve código porque *não consegue*).
- `tools: omitido` = o agente herda todas as tools disponíveis a subagentes. Quase nunca é o que você quer.
- `disallowedTools:` = denylist sobre o pool herdado (tem precedência se ambos existirem).
- O que subagente nunca tem:** `AskUserQuestion` (não pergunta ao usuário — por isso o padrão "registra premissa e segue"), plan mode (`EnterPlanMode / ExitPlanMode`), `EndConversation` . Criar sub-subagentes é possível até um limite de profundidade (3 por padrão); no limite, a tool `Agent` some.
- A **mensagem final** do subagente é a única coisa que volta ao orquestrador — o pai não vê as tool calls intermediárias. (É por isso que o doc-driven grava tudo em arquivo.)

4. Tools MCP

- Convenção de nome: `mcp__<servidor>__<tool>` (ex.: `mcp__playwright__browser_navigate` , `mcp__github__list_pull_requests`).
- Listar: `claude mcp list` no terminal, ou `/mcp` dentro da sessão.
- No frontmatter de agente: liste pelo nome completo, ou `mcp__<servidor>__*` para todas as tools de um servidor.
- Nas permissões: mesma sintaxe (`mcp__github__*` no allow, `mcp__aws__terminate_instance` no deny).

5. Permissões em uma página

Sintaxe das regras — `Tool(especificador)` :

Regra	Vale para	Exemplo
<code>Bash(prefixo *)</code>	Bash	<code>Bash(npm run test *)</code>
<code>Read(caminho/**)</code>	Read, Grep, Glob	<code>Read(~/segredos/**)</code> no deny
<code>Edit(caminho/**)</code>	Edit, Write, NotebookEdit	<code>Edit(src/**)</code>
<code>WebFetch(domain:host)</code>	WebFetch	<code>WebFetch(domain:github.com)</code>
<code>Agent(nome)</code>	Agent	<code>Agent(reviewer)</code>
<code>mcp__server__tool</code>	MCP	<code>mcp__github__*</code>

Precedência: `deny` > `ask` > `allow` — um deny bloqueia mesmo com allow; um ask força prompt mesmo com allow.

Modos (mudam o quanto se pergunta): `default` (manual — só leitura passa direto), `acceptEdits` (edits e file-ops liberados), `plan` (só exploração; edição bloqueada até aprovar o plano), `auto` (quase tudo, com checagens de segurança), `dontAsk` (só o que está no allow), `bypassPermissions` (tudo — só em container/VM isolada).

6. Situacionais e avançadas (existem conforme a instalação/versão)

Tool	Uma linha
<code>LSP</code>	Inteligência de código: ir-para-definição, referências, erros de tipo
<code>Monitor</code>	Acompanha processo/log em background e reage a padrões
<code>EnterWorktree</code> / <code>ExitWorktree</code>	Isola o trabalho num git worktree descartável
<code>Workflow</code>	Orquestração determinística de vários subagentes (fan-out/pipeline)
<code>Artifact</code>	Publica HTML/Markdown como página privada em claude.ai
<code>SendUserFile</code>	Envia um arquivo gerado para o dispositivo do usuário
<code>CronCreate</code> / <code>CronList</code> / <code>CronDelete</code>	Agendamentos recorrentes na sessão
<code>SendMessage</code> / <code>ListAgents</code>	Mensagens entre sessões/agentes
<code>ToolSearch</code>	Carrega sob demanda tools MCP "adormecidas" (deferred)

7. Limites que valem conhecer

- **Bash**: timeout padrão 2 min, máximo 10 min; saída inline ~30k caracteres (o resto fica em arquivo).
- **Read**: pagina com `offset` / `limit`; PDFs grandes vão por faixas de páginas (máx. 20 por chamada).
- **Grep**: respeita `.gitignore`; use `mode: "content"` para ver as linhas, `head_limit` / `offset` para paginar.
- **Glob**: retorna até 100 arquivos, ordenados por modificação recente.
- **Edit**: exige `Read` prévio do arquivo na conversa; `old_string` ambíguo (2+ ocorrências) falha — use `replace_all` conscientemente.
- **WebSearch**: até 200 chamadas por sessão (subagentes contam junto).
- **Subagentes**: até 20 concorrentes; profundidade de spawn padrão 3.

Fontes (doc oficial): [tools-reference](#) · [permissions](#) · [permission-modes](#) · [mcp](#) · [sub-agents](#). A lista de tools evolui a cada versão — confira a doc para o estado atual.